MySQL Trigger Practice Project

Project Summary

This project is a hands-on MySQL trigger practice project built around a product inventory management database. The main goal of this project was to strengthen my understanding of database automation, audit logging, and business rule implementation using MySQL triggers.

The project includes a product database with multiple categories and over 100 sample records. I practiced creating different types of triggers such as `BEFORE INSERT`, `BEFORE UPDATE`, `AFTER UPDATE`, and `BEFORE DELETE` to handle real-world business scenarios.

Some of the key tasks implemented in this project include:

- Tracking stock and price changes automatically
- Creating audit logs for inventory updates
- Preventing invalid data entry
- Blocking unwanted delete operations
- Managing low-stock alerts
- Recording price change percentages
- Applying business rules automatically through triggers

This project helped me improve my practical SQL skills and understand how triggers can be used in real-world systems for data validation, automation, and monitoring.

Skills Practiced

- MySQL Triggers
- Audit Logging
- Data Validation
- Inventory Database Design
- SQL Debugging
- Business Logic Implementation
- Database Automation

Project Goal

The purpose of this project was not only to practice SQL syntax, but also to develop problem-solving skills by implementing real-life database scenarios commonly used in inventory and business management systems.

Database Setup

```
CREATE DATABASE trigger_practice;

USE trigger_practice;
```

Creating Tables

The project uses multiple tables to simulate a product inventory management system.

Products Table

This table stores the main product information including product name, category, price, and stock quantity.

```sql
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2),
    stock INT
);
```

Stock Audit Table

This table stores stock update history whenever product stock changes.

```sql
CREATE TABLE stock_audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    old_stock INT,
    new_stock INT,
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Price Audit Table

This table stores price update history whenever product prices are modified.

```sql
CREATE TABLE price_audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    old_price DECIMAL(10,2),
    new_price DECIMAL(10,2),
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Inserting Sample Data

After creating the tables, I inserted sample product data into the `products` table. The dataset includes multiple product categories such as Electronics, Clothing,

Grocery, Furniture, Sports, Beauty, Toys, Kitchen, and Health.

```sql
INSERT INTO products (product_name, category, price, stock)
VALUES
('Laptop A', 'Electronics', 75000, 20),
('Laptop B', 'Electronics', 82000, 15),
('Mouse A', 'Electronics', 800, 100),
('Keyboard A', 'Electronics', 1500, 80),
('Monitor A', 'Electronics', 18000, 25),
('Phone A', 'Electronics', 45000, 30),
('Phone B', 'Electronics', 52000, 22),
('Headphone A', 'Electronics', 2500, 60),
('Speaker A', 'Electronics', 3500, 45),
('Router A', 'Electronics', 2800, 40),
('T-Shirt A', 'Clothing', 600, 120),
('T-Shirt B', 'Clothing', 700, 110),
('Jeans A', 'Clothing', 1800, 70),
('Jeans B', 'Clothing', 2200, 65),
('Shirt A', 'Clothing', 1200, 90),
('Shirt B', 'Clothing', 1400, 85),
('Jacket A', 'Clothing', 3500, 35),
('Jacket B', 'Clothing', 4200, 25),
('Shoe A', 'Clothing', 3000, 50),
('Shoe B', 'Clothing', 3800, 45),
('Rice 5kg', 'Grocery', 450, 200),
('Rice 10kg', 'Grocery', 850, 150),
('Oil 1L', 'Grocery', 180, 300),
('Oil 5L', 'Grocery', 850, 120),
('Sugar 1kg', 'Grocery', 120, 250),
('Salt 1kg', 'Grocery', 40, 400),
('Flour 2kg', 'Grocery', 160, 220),
('Tea 500g', 'Grocery', 300, 180),
('Coffee 200g', 'Grocery', 450, 100),
('Milk Powder', 'Grocery', 700, 90),
('Book A', 'Stationery', 350, 75),
('Book B', 'Stationery', 420, 70),
('Pen A', 'Stationery', 15, 500),
```

```
('Pen B', 'Stationery', 20, 450),
('Notebook A', 'Stationery', 80, 300),
('Notebook B', 'Stationery', 120, 250),
('Pencil A', 'Stationery', 10, 600),
('Eraser A', 'Stationery', 8, 550),
('Marker A', 'Stationery', 60, 180),
('File A', 'Stationery', 50, 200),
('Chair A', 'Furniture', 2500, 35),
('Chair B', 'Furniture', 3200, 30),
('Table A', 'Furniture', 5500, 20),
('Table B', 'Furniture', 7500, 15),
('Sofa A', 'Furniture', 25000, 8),
('Sofa B', 'Furniture', 32000, 6),
('Bed A', 'Furniture', 28000, 10),
('Bed B', 'Furniture', 35000, 7),
('Wardrobe A', 'Furniture', 18000, 12),
('Wardrobe B', 'Furniture', 22000, 9),
('Football', 'Sports', 900, 60),
('Cricket Bat', 'Sports', 2500, 30),
('Cricket Ball', 'Sports', 250, 100),
('Badminton Racket', 'Sports', 1200, 40),
('Shuttlecock', 'Sports', 100, 200),
('Tennis Racket', 'Sports', 3500, 20),
('Basketball', 'Sports', 1500, 35),
('Dumbbell 5kg', 'Sports', 1200, 45),
('Yoga Mat', 'Sports', 850, 55),
('Skipping Rope', 'Sports', 300, 90),
('Face Wash', 'Beauty', 350, 100),
('Shampoo', 'Beauty', 500, 120),
('Conditioner', 'Beauty', 550, 90),
('Body Lotion', 'Beauty', 650, 80),
('Perfume A', 'Beauty', 1800, 40),
('Perfume B', 'Beauty', 2200, 35),
('Lipstick A', 'Beauty', 700, 70),
('Lipstick B', 'Beauty', 850, 65),
('Cream A', 'Beauty', 900, 60),
('Cream B', 'Beauty', 1100, 55),
('Toy Car', 'Toys', 450, 90),
```

```
('Doll A', 'Toys', 700, 75),
('Puzzle A', 'Toys', 350, 100),
('Blocks A', 'Toys', 900, 60),
('Remote Car', 'Toys', 1800, 35),
('Drone Toy', 'Toys', 3500, 15),
('Teddy Bear', 'Toys', 1200, 50),
('Board Game', 'Toys', 1500, 40),
('Action Figure', 'Toys', 800, 70),
('Toy Train', 'Toys', 2000, 25),
('Plate Set', 'Kitchen', 1200, 60),
('Glass Set', 'Kitchen', 900, 80),
('Knife Set', 'Kitchen', 1500, 45),
('Frying Pan', 'Kitchen', 1800, 35),
('Pressure Cooker', 'Kitchen', 3500, 25),
('Rice Cooker', 'Kitchen', 4500, 20),
('Blender', 'Kitchen', 3000, 30),
('Kettle', 'Kitchen', 2200, 40),
('Water Bottle', 'Kitchen', 500, 100),
('Lunch Box', 'Kitchen', 650, 90),
('Medicine A', 'Health', 120, 300),
('Medicine B', 'Health', 180, 250),
('Vitamin C', 'Health', 350, 150),
('Bandage', 'Health', 80, 200),
('Thermometer', 'Health', 600, 70),
('Mask Pack', 'Health', 150, 180),
('Sanitizer', 'Health', 250, 160),
('First Aid Box', 'Health', 1200, 45),
('Pain Relief Spray', 'Health', 400, 90),
('Digital BP Machine', 'Health', 3500, 20);
```

Trigger Practice Questions & Solutions

Basic Trigger Questions.

1. Create a trigger that saves old and new stock in `stock_audit` whenever product stock is updated.

Solution

```
DELIMITER $$

CREATE TRIGGER product_stock_automator
AFTER UPDATE
ON products
FOR EACH ROW
BEGIN
    INSERT INTO stock_audit(product_id, old_stock, new_stoc
k)
    VALUES(OLD.product_id, OLD.stock, NEW.stock);
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET stock = 90
WHERE product_id = 1;
```

The trigger automatically stores the previous stock and updated stock values into the `stock_audit` table.

2. Create a trigger that saves old and new price in `price_audit` whenever product price is updated.

Solution

```
DELIMITER $$

CREATE TRIGGER product_price_automator
AFTER UPDATE
ON products
FOR EACH ROW
```

```
BEGIN
    INSERT INTO price_audit(product_id, old_price, new_pric
e)
    VALUES(OLD.product_id, OLD.price, NEW.price);
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET price = 1800.00
WHERE product_id = 4;
```

Result

The trigger automatically records both the previous and updated product prices in the `price_audit` table whenever a product price is modified. This helps maintain price change history and supports audit tracking within the inventory system.

3. Create a trigger that prevents inserting a product with negative stock.

Solution

```
DELIMITER $$

CREATE TRIGGER prevent_negative_stock
BEFORE INSERT
ON products
FOR EACH ROW
BEGIN
    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'STOCK CAN NOT BE NEGATIVE';
    END IF ;
```

```
END $$

DELIMITER ;
```

Test Query

```
INSERT INTO products (product_name, stock) VALUES ('Lapto
p', -5);
```

Result

The trigger prevents products from being inserted with negative stock values by generating an error message before the data is saved. This helps maintain data accuracy and ensures inventory records remain valid and reliable.

4. Create a trigger that prevents updating product price to less than 1.

Solution

```
DELIMITER $$

CREATE TRIGGER prevent_update_negative_price
BEFORE UPDATE
ON products
FOR EACH ROW
BEGIN
    IF NEW.price < 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Price cannot be less than 1';
    END IF;
END $$

DELIMITER ;
```

Test Query

```sql
UPDATE products
SET price = -50000
WHERE product_id = 6;
```

Result

The trigger prevents product prices from being updated to invalid values less than 1 by displaying an error message before the update is applied. This helps maintain accurate pricing data and protects the integrity of the inventory system.

5. Create a trigger that automatically sets stock to 0 if inserted stock is negative.

Solution

```sql
DELIMITER $$

CREATE TRIGGER stock_automator
BEFORE INSERT
ON products
FOR EACH ROW
BEGIN
    IF NEW.stock < 0 THEN
    SET NEW.stock = 0;
    END IF;
END $$

DELIMITER ;
```

Test Query

```sql
INSERT INTO products (product_name, stock) VALUES ('Pencil
A', -5);
```

Result

The trigger automatically changes negative stock values to `0` before inserting the product into the database. This helps prevent invalid inventory data and ensures stock records remain consistent and usable.

Intermediate Trigger Questions

6. Create a trigger that prevents deleting products from the `products` table.

Solution

```
DELIMITER $$

CREATE TRIGGER prevent_deleting_products
BEFORE DELETE
ON products
FOR EACH ROW
BEGIN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'DELETING PRODUCTS IS NOT ALLOWED';
END $$

DELIMITER ;
```

Test Query

```
DELETE FROM products WHERE product_id = 6;
```

Result

The trigger blocks all delete operations from the `products` table by displaying an error message before deletion occurs. This helps protect important inventory data from accidental or unauthorized removal.

7. Create a trigger that increases price by 10% automatically if category is Electronics.

Solution

```
DELIMITER $$

CREATE TRIGGER price_increase_automator
BEFORE INSERT
ON products
FOR EACH ROW
BEGIN
    if NEW.category = 'Electronics' THEN
        SET NEW.price = NEW.price + (0.10 * NEW.price);
    END if;
END $$

DELIMITER ;
```

Test Query

```
INSERT INTO products (product_name, category, price)
VALUES ('Keyboard A', 'Electronics', 1000);
```

Result

The trigger automatically increases product prices by 10% before insertion when the product category is `Electronics`. This demonstrates how triggers can be used to apply automated business rules and pricing strategies within a database system.

8. Create a trigger that records price changes only when the new price is greater than the old price.

Solution

```
DELIMITER $$

CREATE TRIGGER price_record_automator
AFTER UPDATE
ON products
FOR EACH ROW
BEGIN
    IF NEW.price > OLD.price THEN
        INSERT INTO price_audit(product_id,old_price,new_pr
ice)
        VALUES(OLD.product_id,OLD.price,NEW.price);
    END if;
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET price = 84000
WHERE product_id = 2;
```

Result

The trigger records price updates in the `price_audit` table only when the new product price is higher than the previous price. This helps focus on price increase tracking and reduces unnecessary audit records for unchanged or reduced prices.

9. Create a trigger that prevents stock update if new stock is greater than 1000.

Solution

```
DELIMITER $$

CREATE TRIGGER stock_updator
BEFORE UPDATE
ON products
FOR EACH ROW
BEGIN
    IF NEW.stock > 1000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'stock can not be greater than 1
000';
    END IF;
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET stock = 1200
WHERE product_id = 5;
```

Result

The trigger prevents stock values from being updated beyond 1000 by displaying an error message before the update is completed. This helps enforce inventory limits and maintain realistic stock records within the system.

10. Create a trigger that changes product name to uppercase before insert.

Solution

```
DELIMITER $$

CREATE TRIGGER name_automator
```

```
BEFORE INSERT
ON products
FOR EACH ROW
BEGIN
    SET NEW.product_name = UPPER(NEW.product_name);
END $$

DELIMITER ;
```

Test Query

```
INSERT INTO products (product_name, category, price)
VALUES ('laptop', 'Electronics', 1000);
```

Result

The trigger automatically converts product names to uppercase before inserting them into the `products` table. This helps maintain consistent formatting and improves data standardization across the inventory database.

Real-Life Trigger Questions

11. Create a trigger to track every stock change in `stock_audit`.

Solution

```
DELIMITER $$

CREATE TRIGGER stock_tracker
AFTER UPDATE
ON products
FOR EACH ROW
BEGIN
    INSERT INTO stock_audit( product_id, old_stock, new_sto
ck)
    VALUES( NEW.product_id, OLD.stock, NEW.stock);
```

```
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET stock = 450
WHERE product_id = 5;
```

Result

The trigger automatically tracks every stock update by storing both the previous and updated stock values in the `stock_audit` table. This helps maintain a complete history of inventory changes for monitoring and audit purposes.

12. Create a trigger to track every price change in `price_audit`.

Solution

```
DELIMITER $$

CREATE TRIGGER price_tracker
AFTER UPDATE
ON products
FOR EACH ROW
BEGIN
    INSERT INTO price_audit( product_id, old_price, new_pri
ce)
    VALUES( NEW.product_id, OLD.price, NEW.price);
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET price = 3200
WHERE product_id = 8;
```

Result

The trigger automatically records every product price update by storing both the previous and updated prices in the `price_audit` table. This helps maintain a complete history of price changes for tracking, monitoring, and audit purposes.

13. Create a trigger that blocks price reduction greater than 50%.

Solution

```
DELIMITER $$

CREATE TRIGGER price_reduction_tracker
BEFORE UPDATE
ON products
FOR EACH ROW
BEGIN
    IF NEW.price < (0.5 * OLD.price) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Suspicious price reduction';
    END IF;
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET price = 30000
WHERE product_id = 2;
```

Result

The trigger prevents product prices from being reduced by more than 50% by displaying an error message before the update is applied. This helps protect the system from suspicious pricing changes and maintains pricing consistency within the inventory database.

14. Create a trigger that prevents product deletion if stock is greater than 0.

```
DELIMITER $$

CREATE TRIGGER product_deletion_tracker
BEFORE DELETE
ON products
FOR EACH ROW
BEGIN
    IF OLD.stock > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete product with rema
ining stock';
    END IF;
END $$

DELIMITER ;
```

Test Query

```
DELETE FROM products WHERE product_id = 2;
```

Result

The trigger prevents products from being deleted when stock is still available by displaying an error message before deletion occurs. This helps avoid accidental loss of active inventory records and ensures better inventory management.

15. Create a trigger that automatically sets category to Uncategorized if category is NULL.

Solution

```
DELIMITER $$

CREATE TRIGGER category_identifier
BEFORE INSERT
ON products
FOR EACH ROW
BEGIN
    IF NEW.category IS NULL THEN
        SET NEW.category = 'Uncategorized';
    END IF;
END $$

DELIMITER ;
```

Test Query

```
INSERT INTO products (product_name, price)
VALUES ('Table', 500);
```

Result

The trigger automatically assigns the category `Uncategorized` when no category value is provided during product insertion. This helps maintain consistent product classification and prevents missing category data in the inventory system.

16. Whenever stock becomes less than 10, insert that product into `low_stock_alerts`.

Creating table `low_stock_alerts`

```sql
CREATE TABLE low_stock_alerts
(
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT,
    product_name VARCHAR(100),
    stock INT,
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Solution

```sql
DELIMITER $$

CREATE TRIGGER low_stock_identifier
AFTER UPDATE
ON products
FOR EACH ROW
BEGIN
    IF NEW.stock < 10 THEN
        INSERT INTO low_stock_alerts( product_id, product_n
ame, stock)
        VALUES(NEW.product_id, NEW.product_name, NEW.stoc
k);
    END IF;
END $$

DELIMITER ;
```

Test Query

```sql
UPDATE products
SET stock = 8
WHERE product_id = 8;
```

Result

The trigger automatically inserts product information into the `low_stock_alerts` table whenever the stock quantity becomes less than 10 after an update. This helps identify low inventory items quickly and supports better inventory monitoring and restocking management.

17. Before deleting any product, save its information into `deleted_products`.

Creating table `deleted_products`

```
CREATE TABLE deleted_products
(
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT,
    product_name VARCHAR(100),
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Solution

```
DELIMITER $$

CREATE TRIGGER delete_logger
BEFORE DELETE
ON products
FOR EACH ROW
BEGIN
        INSERT INTO deleted_products( product_id, product_n
ame)
        VALUES(OLD.product_id, OLD.product_name);
END $$

DELIMITER ;
```

Test Query

```sql
DELETE FROM products
WHERE product_id = 2;
```

Result

The trigger automatically stores deleted product information in the `deleted_products` table whenever a product is removed from the `products` table. This helps maintain a backup record of deleted items and supports audit tracking within the inventory system.

18. After inserting a product, update total product count.

Creating table `product_summary`

```sql
CREATE TABLE product_summary
(
    id INT AUTO_INCREMENT PRIMARY KEY,
    total_product INT,
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP
);


INSERT INTO product_summary(total_product)
VALUES(0);
```

Solution

```sql
DELIMITER $$

CREATE TRIGGER product_counter
AFTER INSERT
ON products
FOR EACH ROW
BEGIN
    UPDATE product_summary
```

```
    SET total_product = total_product + 1;
END $$

DELIMITER ;
```

Test Query

```
INSERT INTO products( product_name, category, price, stock)
VALUES('Mouse A', 'Electronics', 1000, 15);
```

Result

The trigger automatically updates the total product count in the `product_summary` table whenever a new product is inserted into the `products` table. This helps maintain an up-to-date summary of total inventory records without requiring manual updates.

19. Create a trigger that prevents duplicate product names.

Solution

```
DELIMITER $$

CREATE TRIGGER duplicate_product_detector
BEFORE INSERT
ON products
FOR EACH ROW
BEGIN
    IF EXISTS
        (SELECT
            *
        FROM products
        WHERE product_name = NEW.product_name) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Duplicate product name';
    END IF;
```

```
END $$

DELIMITER ;
```

Test Query

```
INSERT INTO products( product_name, category, price, stock)
VALUES('Mouse A', 'Electronics', 1000, 15);
```

Result

The trigger prevents duplicate product names from being inserted into the `products` table by displaying an error message when a matching product name already exists. This helps maintain unique product records and improves data consistency within the inventory system.

20. Create a trigger that stores price change percentage in a new audit table.

Creating table `price_change_log`

```
CREATE TABLE price_change_log
(
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT,
    product_name VARCHAR(100),
    old_price DECIMAL(10,2),
    new_price DECIMAL(10,2),
    price_change_percentage DECIMAL(10,2),
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Solution

```
DELIMITER $$
```

```
CREATE TRIGGER price_change_detector
AFTER UPDATE
ON products
FOR EACH ROW
BEGIN
    IF OLD.price <> NEW.price
    THEN INSERT INTO price_change_log
    (
        product_id,
        product_name,
        old_price,
        new_price,
        price_change_percentage
    )
    VALUES
    (
        NEW.product_id,
        NEW.product_name,
        OLD.price,
        NEW.price,
        ROUND(((NEW.price - OLD.price) / OLD.price) * 100,
2)
    );
END IF;
END $$

DELIMITER ;
```

Test Query

```
UPDATE products
SET price = 6500
WHERE product_id = 1;
```

Result

The trigger automatically calculates and stores the percentage of price change whenever a product price is updated. The previous price, updated price, and calculated percentage are recorded in the `price_change_log` table, helping track pricing trends and monitor product price adjustments more effectively.

GitHub Repository

https://github.com/Rifat250/mysql-trigger-practice-project